# Verified compiler for language with control effects

(Weryfikacja kompilatora dla języka programowania z efektami sterowania)

Paweł Dybiec

Praca magisterska

**Promotor:** dr Piotr Polesiuk

Uniwersytet Wrocławski
Wydział Matematyki i Informatyki
Instytut Informatyki

18 lutego 2026

**Abstract**

We present type system for language with labeled delimited control operators. It utilizes polymorphic types and effects to allow expressions parametrized by effects. Our goal is to formally prove correctness of such language and present translation of it to simple abstract machine.

---

Tematem tej pracy jest stworzenie i zweryfikowanie języka z ograniczonymi operatorami kontroli. Używa on polimorficznych typów aby pozwolić na wyrażenie obliczeń generycznych wobec pewnych efektów. Celem pracy jest zweryfikowanie poprawnosci definicji takiego języka oraz jego translacji do maszyny abstrakcyjnej.

# Contents

# Chapter 1

# Introduction

Control operators have been present in languages for a long time, one noticable example is SCHEME programming language which started in 70s[1]. Operators for delimited control introduced by Danvy, Filinski[2] and Felleisen[3] allow to control them more precisely. They still allow for non-local execution, which enables intresting computational effects such as emulating non-determinism, exceptions, failure and others.

In this paper we try to build typed version of such calculus with polymorphic types (similiar to Piróg, Polesiuk and Sieczkowski[4]) and named labels. Usually such calculi jump from shift to closest enclosing reset. Here we tried labeling them to allow stacking of different effects. This was planned originally to use approach similiar to described by Materzok, Biernacki[5] to compile language to abstract machine.

Formalisation work was done in Agda 2.8. Code archive has also nix derivation describing exact environment to build both package and this paper to ensure reproducibility.

# Chapter 2

# Types

In this section we define syntax of the base language and types.

## 2.1 Kinds, Types, Effects and Expressions

Variable and type variables are de Bruijn indices.

```
data Kind : Set where
  T : Kind
  E : Kind
Id : Set
Id = ℕ
```

Types and Effects are defined mutally recursive. Type is either variable, arrow, forall or label. Effect is list of types, it's stored in arrow and forall constructor to keep effects of computation underneath. Label type just stores Type and Effect of other computation.

```
module Types where
  data Type : Set
  Effects : Set
  data Type where
    ttv : Id → Type
    _-_>_ : Type → Effects → Type → Type
    forallt : Kind → Type → Effects → Type
    L_at_/_ : Type → Type → Effects → Type
  Effects = List Type
open Types
```

`var`, `lam`, `app`, `tlam`, `tapp` are behaving as expected, with the only exception being that `tlam` holds kind of parameter. `new` bind label that is used by next constructions to pair up. $\texttt{shift}_0$ when evaluated finds $\texttt{reset}_0$ is handling same label, continuation between them is captured (including reset) and passed into computation under shift as variable.

Type substitution in types is removed for brevity. It's available in accompanying repository.

```
data Expr : Set where
  var : ℕ → Expr
```

```
  lam : Expr → Expr
  app : Expr → Expr → Expr
  tlam : Kind → Expr → Expr
  tapp : Expr → Type → Expr
  new : Expr → Expr
  shift₀ : Expr → Expr → Expr
  reset₀ : Expr → Expr → Expr → Expr
```

## 2.2 Contexts

Contexts are represented as list of types, and type contexts are represented as lists of kinds. Judgements for membership of types have Peano numbers structure.

```
infixl 5 _,_
data Context : Set where
    ∅ : Context
    _,_ : Context → Type → Context

data TContext : Set where
    ∅ : TContext
    _,_ : TContext → Kind → TContext

infix 4  _∋_⦂_
data _∋_⦂_ : Context → Id → Type → Set where
  Z : ∀ {Γ A}
    → (Γ , A) ∋ zero ⦂ A

  S : ∀ {Γ x y A}
    → Γ ∋ x ⦂ A
    → (Γ , y) ∋ (suc x) ⦂ A

data _∋t_⦂_ : TContext → Id → Kind → Set where
  Z : ∀ {Δ k}
    → (Δ , k) ∋t zero ⦂ k

  S : ∀ {Δ x y k}
    → Δ ∋t x ⦂ k
    → (Δ , y) ∋t (suc x) ⦂ k
```

## 2.3 Typing judgements

Expressions are extrinsically typed, thus typing judgements are represented separately.

```
data _⊢_<⦂_ : TContext → Effects → Effects → Set where
  Z : ∀ {Δ}
       → Δ ⊢ nil <⦂ nil
  S : ∀ {Δ e E1 E2 }
       → Δ ⊢ E1 <⦂ E2
       → Δ ⊢ (e :: E1) <⦂ (e :: E2)
  S' : ∀ {Δ e E1 E2 }
       → Δ ⊢ E1 <⦂ E2
       → Δ ⊢ E1 <⦂ (e :: E2)

nil<⦂⊥ : ∀ {Δ E } → Δ ⊢ E <⦂ nil → E ≡ nil
nil<⦂⊥ (Z) = refl

data _⊢_<t⦂_ : TContext → Type → Type → Set where

  <⦂refl : ∀ {Δ A} → Δ ⊢ A <t⦂ A

  <⦂→ : ∀ {Δ A1 A2 B1 B2 E1 E2}
       → Δ ⊢ E1 <⦂ E2
       → Δ ⊢ A1 <t⦂ A2
       → Δ ⊢ B1 <t⦂ B2
```

```
      → Δ ⊢ (A2 - E1 > B1) <t: (A1 - E2 > B2)

  <:forall : ∀ {Δ A1 A2 k E1 E2}
      → (Δ , k) ⊢ A1 <t: A2
      → Δ ⊢ E1 <: E2
      → Δ ⊢ forallt k A1 E1 <t: forallt k A2 E2
```

Typing rules for `var`, `lam`, `app`, `tlam`, `tapp` are defined mostly as usual. The only difference is that expressions for values are generic over effect they execute.

```
open TypeSubst
data _,_⊢_:_/_ : TContext → Context → Expr → Type → Effects → Set where
  ⊢var : ∀ {Γ Δ x A E}
    → Γ ∋ x : A
    ----------------------
    → Δ , Γ ⊢ var x : A / E

  ⊢lam : ∀ {Γ Δ e A B E F}
    → Δ , (Γ , A) ⊢ e : B / E
    ---------------------------------
    → Δ , Γ ⊢ lam e : A - E > B / F

  ⊢weak : ∀ {Γ Δ e A A' E E'}
    → Δ ⊢ A <t: A'
    → Δ ⊢ E <: E'
    → Δ , Γ ⊢ e : A / E
    ---------------------
    → Δ , Γ ⊢ e : A' / E'

  ⊢app : ∀ {Γ Δ e1 e2 A B E}
    → Δ , Γ ⊢ e1 : A - E > B / E
    → Δ , Γ ⊢ e2 : A / E
    ---------------------------
    → Δ , Γ ⊢ app e1 e2 : B / E

  ⊢forall : ∀ {Γ Δ e k A E F}
    → (Δ , k) , Γ ⊢ e : bump A / bump' E
    ------------------------------------
    → Δ , Γ ⊢ tlam k e : forallt k A E / F

  ⊢tapp : ∀ {Γ Δ e k A T E}
    → Δ ⊢ T :t
    → Δ , Γ ⊢ e : forallt k A E / E
    ------------------------------------------------
    → Δ , Γ ⊢ tapp e T : A [ T ] / (E effs[t T ])
```

`new` introduces new type variable, and variable that represent effect and label. Type of label stores type of label (here `ttv zero`). `A1` / `A2` represent type and effect of corresponding reset.

```
  ⊢new : ∀ {Γ Δ e A A1 E E1}
    → (Δ , Kind.E) , (Γ , (L ttv zero at A1 / E1)) ⊢ e : bump A / bump' E
    ----------------------
    → Δ , Γ ⊢ new e : A / E
```

$\texttt{shift}_0$ uses only one effect `ttv n` that's represented by label. For it to be properly typed expression inside shift bind extra variable - where continuation will be plugged into. So type of this continuation should be function type from type of shift to reset, with effects visible in reset. Since continuation passed there will have $\texttt{reset}_0$, that $\texttt{reset}_0$ will introduce effect `ttv n`, so it shouldn't be represented in type of argument.

```
  ⊢shift₀ : ∀ {Γ Δ e e' A A' n E'}
    → Δ ⊢ ttv n :e
    → Δ , Γ ⊢ e' : (L ttv n at A' / E') / nil
    → Δ , (Γ , A - E' > A' ) ⊢ e : A' / E'
    ---------------------------------------
```

```
  → Δ , Γ ⊢ shift₀ e' e ꞉ A / (ttv n :: nil)
```

`reset₀` has three parameters, first is expression that will have access to effect, so its list of effects is expanded. Second is continuation that will handle value returned from first argument. And third is label, which type stores type and effects of whole expression.

```
⊢reset₀ : ∀ {Γ Δ e e' en A A' n E'}
  → Δ ⊢ ttv n ꞉e
  → Δ , Γ ⊢ e ꞉ A / (ttv n :: E')
  → Δ , (Γ , A) ⊢ en ꞉ A' / E'
  → Δ , Γ ⊢ e' ꞉ (L ttv n at A' / E') / nil
  ------------------------------------
  → Δ , Γ ⊢ reset₀ e en e' ꞉ A' / E'
```

# Chapter 3

# Runtime

## 3.1 Runtime

Since grammar of terms doesn't have any expression that would have type of label, and shift and reset require same labels, that would mean that reduction relation would need to go under `new` binders. Instead we will define another expression language expanded by notion of label values.

When `new` expression is evaluated then all occurences of variables bound by it would have it replaced with newly allocated label value. That means evaluation would need to keep a state for allocator.

Since `new` binds type variables that represent effects, and those type variables are present in typing judgements of subexpressions, complete removal of `new` during evaluation would break type preservation. Thus we introduce `new'` that just binds type variables, and stores allocated label.

To make sure that label values bound to same type variable have same values, we change typing context so it stores label bound to that type variable.

```
module Runtime where
```

Most of constructors in `RExpr` are the same as in `Expr`. Labels runtime values are represented by Natural numbers.

```
module RuntimeExpr where
  Label = ℕ
  data RExpr : Set where --runtime version
    var : ℕ → RExpr
    lam : RExpr → RExpr
    app : RExpr → RExpr → RExpr
    tlam : Kind → RExpr → RExpr
    tapp : RExpr → Type → RExpr
    new : RExpr → RExpr
```

We are using `new'` to hold allocation of labels.

```
new' : Label → RExpr → RExpr
shift₀ : RExpr → RExpr → RExpr
reset₀ : RExpr → RExpr → RExpr → RExpr
```

And here we have separate term for labels, it just stores label identifier.

```
label : Label → RExpr
```

Typing Context now has different representation, because we store labels for typing variables bound by `new'`. Thus all of judgements using them need to be redefined for the runtime language.

```
data TContext : Set where
  ∅ : TContext
  `t : TContext → TContext
  `e : Data.Maybe.Maybe Label → TContext → TContext
push : Kind → TContext → TContext
push E xs = `e Data.Maybe.nothing xs
push T xs = `t xs
data _∋t_⦂_ : TContext → Id → Kind → Set where
  Zt : ∀ {Δ }
       → `t Δ ∋t zero ⦂ T

  Ze : ∀ {Δ l }
       → `e l Δ  ∋t zero ⦂ E

  St : ∀ {Δ x k}
       → Δ ∋t x ⦂ k
       → `t Δ ∋t (suc x) ⦂ k
  Se : ∀ {Δ x l k}
       → Δ ∋t x ⦂ k
       → `e l Δ ∋t (suc x) ⦂ k

data _∋l_⦂_ : TContext → Id → Label → Set where
  Z : ∀ {Δ l }
       → `e (Data.Maybe.just l) Δ ∋l zero ⦂ l
  St : ∀ {Δ x l}
       → Δ ∋l x ⦂ l
       → `t Δ ∋l (suc x) ⦂ l
  Se : ∀ {Δ x l l'}
       → Δ ∋l x ⦂ l
       → `e l' Δ ∋l (suc x) ⦂ l
data _⊢_⦂e : TContext → Type → Set where
  ⊢ttv : ∀ {Δ n}
       → Δ ∋t n ⦂ E
       → Δ ⊢ ttv n ⦂e


data _⊢_<⦂_ : TContext → Effects → Effects → Set where
  Z : ∀ {Δ}
       → Δ ⊢ nil <⦂ nil
  S : ∀ {Δ e E1 E2 }
       → Δ ⊢ E1 <⦂ E2
       → Δ ⊢ (e :: E1) <⦂ (e :: E2)
  S' : ∀ {Δ e E1 E2 }
       → Δ ⊢ E1 <⦂ E2
       → Δ ⊢ E1 <⦂ (e :: E2)
<⦂e-refl : ∀ {Δ E} → Δ ⊢ E <⦂ E
<⦂e-refl {Δ} {nil} = Z
<⦂e-refl {Δ} {x :: E₁} = S <⦂e-refl

data _⊢_<t⦂_ : TContext → Type → Type → Set where
  <⦂refl : ∀ {Δ A} → Δ ⊢ A <t⦂ A
  <⦂→ : ∀ {Δ A1 A2 B1 B2 E1 E2}
       → Δ ⊢ E1 <⦂ E2
       → Δ ⊢ A1 <t⦂ A2
       → Δ ⊢ B1 <t⦂ B2
       → Δ ⊢ (A2 - E1 > B1) <t⦂ (A1 - E2 > B2)
  <⦂forall : ∀ {Δ A1 A2 k E1 E2}
       → (push k Δ) ⊢ A1 <t⦂ A2
       → Δ ⊢ E1 <⦂ E2
       → Δ ⊢ forallt k A1 E1 <t⦂ forallt k A2 E2

module RExprSubst where
```

Substitution of types in expressions is needed for evaluation of type application.

```
substT-in-rexpr : TypeSubst.Subst → RExpr → RExpr
substT-in-rexpr ρ (tlam k e) = tlam k (substT-in-rexpr (TypeSubst.exts ρ) e)
```

```
substT-in-rexpr ρ (new e) = new (substT-in-rexpr (TypeSubst.exts ρ) e)
substT-in-rexpr ρ (new' l e) = new' l (substT-in-rexpr (TypeSubst.exts ρ) e)

substT-in-rexpr ρ (tapp e t) = tapp (substT-in-rexpr ρ e) (TypeSubst.subst ρ t)
substT-in-rexpr ρ (var x) = var x
substT-in-rexpr ρ (lam e) = lam (substT-in-rexpr ρ e)
substT-in-rexpr ρ (app e e₁) = app (substT-in-rexpr ρ e) (substT-in-rexpr ρ e₁)
substT-in-rexpr ρ (shift₀ e e₁) = shift₀ (substT-in-rexpr ρ e) (substT-in-rexpr ρ e₁)
substT-in-rexpr ρ (reset₀ e e₁ e₂) = reset₀ (substT-in-rexpr ρ e) (substT-in-rexpr ρ e₁) (substT-in-rexpr ρ e₂)
substT-in-rexpr ρ (label l) = label l

_e[t_] : RExpr → Type → RExpr
M e[t t ] = substT-in-rexpr (TypeSubst.subst-zero t) M
```

Most of typing judgement are working as before.

```
data _;_⊢_⦂_/_ : TContext → Context → RExpr → Type → Effects → Set where

  ⊢var : ∀ {Γ Δ x A E}
    → Γ ∋ x ⦂ A
    ------------------------
    → Δ ; Γ ⊢ var x ⦂ A / E

  ⊢lam : ∀ {Γ Δ e A B E F}
    → Δ ; (Γ , A) ⊢ e ⦂ B / E
    -------------------------------
    → Δ ; Γ ⊢ lam e ⦂ A - E > B / F
  ⊢weak : ∀ {Γ Δ e A A' E E'}
    → Δ ⊢ A <:⦂ A'
    → Δ ⊢ E <⦂ E'
    → Δ ; Γ ⊢ e ⦂ A / E
    ---------------------
    → Δ ; Γ ⊢ e ⦂ A' / E'
  ⊢app : ∀ {Γ Δ e1 e2 A B E}
    → Δ ; Γ ⊢ e1 ⦂ A - E > B / E
    → Δ ; Γ ⊢ e2 ⦂ A / E
    ----------------------------
    → Δ ; Γ ⊢ app e1 e2 ⦂ B / E

  ⊢forall : ∀ {Γ Δ e k A E F}
    → (push k Δ ) ; Γ ⊢ e ⦂ TypeSubst.bump A / TypeSubst.bump' E
    ----------------------------------
    → Δ ; Γ ⊢ tlam k e ⦂ forallt k A E / F

  ⊢tapp : ∀ {Γ Δ e k A T E}
    → Δ ⊢ T ⦂t
    → Δ ; Γ ⊢ e ⦂ forallt k A E / E
    ------------------------------------------------------------------
    → Δ ; Γ ⊢ tapp e T ⦂ A TypeSubst.[ T ] / (E TypeSubst.effs[t T ])

  ⊢new : ∀ {Γ Δ e A A1 E E1}
    → (push Kind.E Δ) ; (Γ , (L ttv zero at A1 / E1)) ⊢ e ⦂ TypeSubst.bump A / TypeSubst.bump' E
    ---------------------
    → Δ ; Γ ⊢ new e ⦂ A / E
```

`new'` stores label id in typing context so it can verify correctness of all label values.

```
⊢new' : ∀ {Γ Δ e l A E l'}
  → (`e (Data.Maybe.just l) Δ) ; Γ ⊢ e ⦂ TypeSubst.bump A / TypeSubst.bump' E
  ---------------------------
  → Δ ; Γ ⊢ new' l' e ⦂ A / E

⊢shift₀ : ∀ {Γ Δ e e' A A' n E'}
  → Δ ⊢ ttv n ⦂e
  → Δ ; Γ ⊢ e' ⦂ (L ttv n at A' / E') / nil
  → Δ ; (Γ , A - E' > A' ) ⊢ e ⦂ A' / E'
  ---------------------------------------
  → Δ ; Γ ⊢ shift₀ e' e ⦂ A / (ttv n :: nil)

⊢reset₀ : ∀ {Γ Δ e e' en A A' n E'}
  → Δ ⊢ ttv n ⦂e
  → Δ ; Γ ⊢ e' ⦂ (L ttv n at A' / E') / nil
  → Δ ; Γ ⊢ e ⦂ A / (ttv n :: E')
  → Δ ; (Γ , A) ⊢ en ⦂ A' / E'
```

```
  ----------------------------------
  → Δ ; Γ ⊢ reset₀ e en e' ∶ A' / E'
```

Best we can do statically here is checking that label value really corresponds. To prove prove safety we need to add extra conditions on label expressions. That is every label with same value needs to have same type (modulo type indices). Such extra condition would have to be passed to progress, and preservation would need to also prove that such condition is kept.

```
⊢label : ∀ {Γ Δ n l A E F}
  → Δ ∋l n ∶ l
  ---------------------------------------------
  → Δ ; Γ ⊢ label l ∶ (L (ttv n) at A / E) / F
```

## 3.2 Runtime embedding

We defined embedding of original language in current one, and proof that such embedding preserves typing judgements. Part of proof is skipped as it goes through almost all judgement types.

```
runtime : Expr → RExpr
runtime (var x) = var x
runtime (lam x) = lam (runtime x)
runtime (app x x₁) = app (runtime x) (runtime x₁)
runtime (tlam x x₁) = tlam x (runtime x₁)
runtime (tapp x x₁) = tapp (runtime x) x₁
runtime (new x) = new (runtime x)
runtime (shift₀ x x₁) = shift₀ (runtime x) (runtime x₁)
runtime (reset₀ x x₁ x₂) = reset₀ (runtime x) (runtime x₁) (runtime x₂)
runtimeΔ : Types.TContext → TContext
runtimeΔ ∅ = ∅
runtimeΔ (Δ , k) = push k (runtimeΔ Δ)



runtime-types : ∀ {Δ Γ e T E}
  → Δ , Γ ⊢ e ∶ T / E → (runtimeΔ Δ ; Γ ⊢ (runtime e) ∶ T / E)
runtime-types (⊢var x) = ⊢var x
runtime-types (⊢lam t) = ⊢lam (runtime-types t)
runtime-types (⊢weak x x₁ t) = ⊢weak (runtime<t∶ x) (runtime<∶ x₁) (runtime-types t)
runtime-types (⊢app t t₁) = ⊢app (runtime-types t) (runtime-types t₁)
runtime-types (⊢forall t) = ⊢forall (runtime-types t)
runtime-types (⊢tapp x t) = ⊢tapp (runtime⊢t x) (runtime-types t)
runtime-types (⊢new t) = ⊢new (runtime-types t)
runtime-types (⊢shift₀ x t t₁) = ⊢shift₀ ((runtime⊢e x)) (runtime-types t) (runtime-types t₁)
runtime-types (⊢reset₀ x t t₁ t₂) = ⊢reset₀ (runtime⊢e x) (runtime-types t₂) (runtime-types t) (runtime-types t₁)
```

Small lemma about lifting context and that it keeps type safety.

```
_#_ : Context → Context → Context
y # ∅ = y
y # (xs , x) = (y # xs) , x
∋↑ : ∀ {Γ x A Γ'}
  → Γ ∋ x ∶ A
  → Γ' # Γ ∋ x ∶ A
∋↑ {Γ = ∅} ()
∋↑ {Γ = Γ , x} Z = Z
∋↑ {Γ = Γ , x} (S t) = S (∋↑ t)
e↑ : ∀ {Δ Γ e T E Γ'}
  → Δ ; Γ ⊢ e ∶ T / E
  → Δ ; (Γ' # Γ) ⊢ e ∶ T / E
e↑ (⊢var x) = ⊢var (∋↑ x)
e↑ (⊢lam t) = ⊢lam (e↑ t)
e↑ (⊢weak x x₁ t) = ⊢weak x x₁ (e↑ t)
```

```
e↑ (⊢app t t₁) = ⊢app (e↑ t) (e↑ t₁)
e↑ (⊢forall t) = ⊢forall (e↑ t)
e↑ (⊢tapp x t) = ⊢tapp x (e↑ t)
e↑ (⊢new t) = ⊢new (e↑ t)
e↑ (⊢new' t) = ⊢new' (e↑ t)
e↑ (⊢shift₀ x t t₁) = ⊢shift₀ x (e↑ t) (e↑ t₁)
e↑ (⊢reset₀ x t t₁ t₂) = ⊢reset₀ x (e↑ t) (e↑ t₁) (e↑ t₂)
e↑ (⊢label x) = ⊢label x
```

Substitution and proof relating typing judgement of inputs and result are defined together inductively.

```
module RExprSubstTyped where
  ext : ∀ {Γ Γ' }
      → (∀ {A n } → Γ ∋ n ⦂ A → Σ[ m ∈ ℕ ] Γ' ∋ m ⦂ A)
      → (∀ {A B n} → (Γ , B) ∋ n ⦂ A → Σ[ m ∈ ℕ ] (Γ' , B) ∋ m ⦂ A)
  ext ρ Z = zero ,, Z
  ext ρ (S x) = suc (ρ x .proj₁) ,, S (ρ x .proj₂)

  rename : ∀ {Γ Γ'}
      → (∀ {A n } → Γ ∋ n ⦂ A → Σ[ m ∈ ℕ ] Γ' ∋ m ⦂ A)
      → (∀ {Δ A e E} → Δ ; Γ ⊢ e ⦂ A / E → Σ[ e' ∈ RExpr ] Δ ; Γ' ⊢ e' ⦂ A / E)
  rename ρ (⊢var { x = n } x) = (var (ρ x .proj₁)) ,, (⊢var (ρ x .proj₂) )
  rename ρ (⊢lam x) = (lam (rename (ext ρ) x .proj₁)) ,, (⊢lam (proj₂ (rename (ext ρ) x) ) )
  rename ρ (⊢weak x x₁ x₂ ) = (rename ρ x₂ .proj₁) ,, ⊢weak x x₁ (rename ρ x₂ .proj₂)
  rename ρ (⊢app x x₁) = app (rename ρ x .proj₁) (rename ρ x₁ .proj₁) ,, ⊢app (rename ρ x .proj₂) (rename ρ x₁ .proj₂)
  rename ρ (⊢forall {k = k} x) = tlam k (rename ρ x .proj₁) ,, ⊢forall (rename ρ x .proj₂)
  rename ρ (⊢tapp x x₁) = tapp (rename ρ x₁ .proj₁) _ ,, ⊢tapp x (rename ρ x₁ .proj₂)
  rename ρ (⊢new x) = new (rename (ext ρ) x .proj₁) ,, ⊢new (rename (ext ρ) x .proj₂)
  rename ρ (⊢new' {l = l} x) = new' l (rename ρ x .proj₁)
    ,, ⊢new' (rename ρ x .proj₂)
  rename ρ (⊢shift₀ x x₁ x₂) = shift₀ (rename ρ x₁ .proj₁) ( rename (ext ρ) x₂ .proj₁)
    ,, ⊢shift₀ x (rename ρ x₁ .proj₂) (rename (ext ρ) x₂ .proj₂)
  rename ρ (⊢reset₀ x x₁ x₂ x₃) = reset₀ (rename ρ x₂ .proj₁) (rename (ext ρ) x₃ .proj₁) (rename ρ x₁ .proj₁)
    ,,
    ⊢reset₀ x (rename ρ x₁ .proj₂) (rename ρ x₂ .proj₂) (rename (ext ρ) x₃ .proj₂)
  rename ρ (⊢label x ) = _ ,, ⊢label x
  postulate
      pushΔ : ∀ {Γ Γ' Δ k}
           → (∀ {n A E} → Γ ∋ n ⦂ A → Σ[ e ∈ RExpr ] Δ ; Γ' ⊢ e ⦂ A / E)
           → (∀ {n A E} → Γ ∋ n ⦂ A → Σ[ e ∈ RExpr ] (push k Δ) ; Γ' ⊢ e ⦂ A / E)
      eΔ : ∀ {Γ Γ' Δ k}
           → (∀ {n A E} → Γ ∋ n ⦂ A → Σ[ e ∈ RExpr ] Δ ; Γ' ⊢ e ⦂ A / E)
           → (∀ {n A E} → Γ ∋ n ⦂ A → Σ[ e ∈ RExpr ] (`e k Δ) ; Γ' ⊢ e ⦂ A / E)
  exts : ∀ {Γ Γ' Δ}
      → (∀ {n A E} → Γ ∋ n ⦂ A → Σ[ e ∈ RExpr ] Δ ; Γ' ⊢ e ⦂ A / E)
      → (∀ {n A B E} → Γ , B ∋ n ⦂ A → Σ[ e ∈ RExpr ] Δ ; Γ' , B ⊢ e ⦂ A / E)
  exts ρ Z = (var zero) ,, (⊢var Z)
  exts ρ (S x) = rename (λ {A = A₁} {n} z → suc n ,, S z) (ρ x .proj₂)

  subst : ∀ {Δ Γ Γ'}
      → (∀ {n A E} → Γ ∋ n ⦂ A → Σ[ e ∈ RExpr ] Δ ; Γ' ⊢ e ⦂ A / E)
      → (∀ {e A E} → Δ ; Γ ⊢ e ⦂ A / E → Σ[ e ∈ RExpr ] Δ ; Γ' ⊢ e ⦂ A / E)
  subst σ (⊢var x) = σ x
  subst σ (⊢lam x) = lam (subst (exts σ) x .proj₁) ,, ⊢lam (subst (exts σ) x .proj₂)
  subst σ (⊢weak x x₁ x₂) = subst σ x₂ .proj₁ ,, ⊢weak x x₁ (subst σ x₂ .proj₂)
  subst σ (⊢app x x₁) = app (subst σ x .proj₁) (subst σ x₁ .proj₁) ,, ⊢app (subst σ x .proj₂) (subst σ x₁ .proj₂)
  subst σ (⊢forall {k = k} x) = tlam k (subst (pushΔ σ) x .proj₁) ,, ⊢forall (subst (pushΔ σ) x .proj₂)
  subst σ (⊢tapp x x₁) = tapp (subst σ x₁ .proj₁) _ ,, ⊢tapp x (subst σ x₁ .proj₂)
  subst σ (⊢new x) = new (subst (pushΔ (exts σ)) x .proj₁)
    ,, ⊢new (subst (pushΔ (exts σ))  x .proj₂)
  subst σ {e = new' l _}(⊢new' x) = new' l (subst (eΔ σ) x .proj₁)
    ,, ⊢new' (subst (eΔ σ) x .proj₂)
  subst σ (⊢shift₀ x x₁ x₂) = shift₀ (subst σ x₁ .proj₁) (subst (exts σ) x₂ .proj₁)
    ,, ⊢shift₀ x (subst σ x₁ .proj₂) (subst (exts σ) x₂ .proj₂)
  subst σ (⊢reset₀ x x₁ x₂ x₃) = reset₀ (subst σ x₂ .proj₁) ( subst (exts σ) x₃ .proj₁) (subst σ x₁ .proj₁)
    ,, ⊢reset₀ x (subst σ x₁ .proj₂) (subst σ x₂ .proj₂) (subst (exts σ) x₃ .proj₂)
  subst σ (⊢label {l = l} x) = label l ,, ⊢label x

  _[_] : ∀ {Δ Γ A B E1}
      → (e e1 : RExpr)
      → {te : Δ ; Γ , A ⊢ e ⦂ B / E1}
      → {te1 : ∀ {E } → Δ ; Γ ⊢ e1 ⦂ A / E}
      → Σ[ e' ∈ RExpr ] Δ ; Γ ⊢ e' ⦂ B / E1
  _[_] {Δ}{Γ}{A}{B}{E1}e e1 {te}{te1} = subst {Δ}{Γ , A}{Γ} σ te
    where
      σ : (∀ {B n E} → Γ , A ∋ n ⦂ B → Σ[ e ∈ RExpr ] Δ ; Γ ⊢ e ⦂ B / E)
      σ {E = E'} Z = e1 ,, (te1 {E'})
      σ {n = suc n} (S x) = var n ,, ⊢var x
```

# Chapter 4

# Progress

## 4.1 Values

Here we define values. Only abstractions, type abstractions and labels are considered values. Since values themself don't perform any effects, they have `nil` effect. But rules for all of them have built-in weakinging. We can use that to generalize their type and perform substiution where any effect is expected.

```
data Value : RExpr → Set where
  vlam : ∀ { e } → Value (lam e)
  vLam : ∀ { k e } → Value (tlam k e)
  vlab : ∀ { n } → Value (label n)
gvalue : ∀ {Δ Γ T E e} → (Value e) → (Δ ; Γ ⊢ e ∶ T / E) → ∀ {F} → (Δ ; Γ ⊢ e ∶ T / F)
--generalize value to any effect
gvalue vlam (⊢lam t) = ⊢lam t
gvalue vLam (⊢forall t) = ⊢forall t
gvalue vlab (⊢label x) = ⊢label x
gvalue {Δ} v (⊢weak x x₁ x₂) {F} = (⊢weak x ( <∶e-refl {Δ} {F}) (gvalue v x₂))
```

Frames would usally be named evaluation context, but here it's taken by typing context. Here frame represents parts between reset s, or reset  and shift . Frame type is parametrized by Δ Γ - typing context outside of frame, T type of the hole. It's also indexed by Effects and Type of whole frame, Effects of the hole, typing context of the hole and amount of new' constructors - which says how many type binders are in the frame. Frames here are intrinsically typed, thus they also store type judgements of subexpressions.

```
data Frame (Δ : TContext) (Γ : Context) (T : Type) : Effects → Type → Effects → TContext → ℕ → Set where
  fempty : ∀ {Eff}
  ------------------------------
    → Frame Δ Γ T Eff T Eff Δ zero
  fapp₁ : ∀ {A B n Δ' Eff E} → Frame Δ Γ T Eff (A - Eff > B) E Δ' n
    → (e : RExpr) → { Δ ; Γ ⊢ e ∶ A / Eff }
    -------------------------
    → Frame Δ Γ T Eff B E Δ' n
  fapp₂ : ∀ {A B n Δ' Eff E} → (e : RExpr) → { v : Value e}
    → { Δ ; Γ ⊢ e ∶ ( A - Eff > B) / Eff }
    ------------------------------------------------------
    → Frame Δ Γ T Eff A E Δ' n → Frame Δ Γ T Eff B E Δ' n
  fnew' : ∀ {A n Δ' Eff E} → (l : Label)
    → Frame (`e (Data.Maybe.just l) Δ) Γ T (TypeSubst.bump' Eff) (TypeSubst.bump A) E Δ' n
    -------------------------------
    → Frame Δ Γ T Eff A E Δ' (suc n)
  freset-label : ∀ {A n Δ' E l' A' Eff}
```

```
    → (e en : RExpr)
    → Δ ⊢ ttv l' ꞉e
    → Δ ; Γ ⊢ e ꞉ A' / (ttv l' :: Eff)
    → Δ ; (Γ , A') ⊢ en ꞉ A /  Eff
    → Frame Δ Γ T nil (L ttv l' at A / Eff) E Δ' n
    -------------------------
    → Frame Δ Γ T Eff A E Δ' n
  fshift-label : ∀ {A n Δ' E l' A' E'}
    → (e : RExpr)
           → Δ ⊢ ttv l' ꞉e
           → Δ ; (Γ , A - E' > A' ) ⊢ e ꞉ A' / E'
    → Frame Δ Γ T nil (L ttv l' at A' / E') E Δ' n
    ------------------------------------
    → Frame Δ Γ T (ttv l' :: nil) A E Δ' n
```

Frame plugging and composition:

```
plug : ∀ {Δ Δ' Γ T Eff A n E}
  → Frame Δ Γ T Eff A E Δ' n
  → (e : RExpr) → Δ' ; Γ ⊢ e ꞉ T / E
  → Σ[ res ∈ RExpr ] (Δ ; Γ ⊢ res ꞉ A / Eff)
_∘f_ : ∀ {Γ Δ Δ' Δ'' Eff Eff' Eff'' A B C n m}
  → Frame Δ Γ B Eff A Eff' Δ' n
  → Frame Δ' Γ C Eff' B Eff'' Δ'' m
  → Frame Δ Γ C Eff A Eff'' Δ'' (n + m)
```

We can prove how plugging and composition relate. Plugging expression into one frame and another results in same value and type as plugging expression into composition of two frames.

```
∘f-lemma : ∀ {Γ Δ Δ' Δ'' Eff Eff' Eff'' A B C n m}
  → (f1 : Frame Δ Γ B Eff A Eff' Δ' n)
  → (f2 : Frame Δ' Γ C Eff' B Eff'' Δ'' m)
  → (e : RExpr) → (t : Δ'' ; Γ ⊢ e ꞉ C / Eff'')
  → plug ( f1 ∘f f2) e t
  ≡ ((λ x → plug f1 (Data.Product.proj₁ x) (Data.Product.proj₂ x))(plug f2 e t))

∘f-lemma fempty f2 e t = refl
∘f-lemma (fapp₁ f1 e₁) f2 e t rewrite ∘f-lemma f1 f2 e t = refl
∘f-lemma (fapp₂ e₁ f1) f2 e t rewrite ∘f-lemma f1 f2 e t = refl
∘f-lemma (fnew' x f1) f2 e t rewrite ∘f-lemma f1 f2 e t = refl
∘f-lemma (freset-label ee en x x₁ x₂ f) f2 e t rewrite ∘f-lemma f f2 e t = refl
∘f-lemma (fshift-label e₁ x x₁ f) f2 e t rewrite ∘f-lemma f f2 e t = refl
```

Lifting frames into arbitrary context preserves types, same as with expressions.

```
↑f : forall { Δ A B Eff Eff' Δ' n Γ' Γ}
  → Frame Δ Γ A Eff B Eff' Δ' n
  → Frame Δ (Γ' ++ Γ) A Eff B Eff' Δ' n
↑f fempty = fempty
↑f (fapp₁ f e {t}) = fapp₁ (↑f f) e {e↑ t}
↑f (fapp₂ e {v} {t} f) = fapp₂ e {v} {e↑ t} (↑f f)
↑f (fnew' l f) = fnew' l (↑f f)
↑f (freset-label e en x x₁ x₂ f) = freset-label e en x (e↑ x₁) (e↑ x₂) (↑f f)
↑f (fshift-label e x x₁ f) = fshift-label e x (e↑ x₁) (↑f f)
```

Metaframe stores whole evaluation context, it's split into frames separated by resets. Type parameters and indices work the same as in frame. Unlike in frame, metaframe now stores resets, so lists of effects inside and outside of frame may differ. That means their difference represents list of effects handled by the frame. This observation can be used to prove that for well typed expression (in empty typing context, and with condition that same labels have the same type) that decomposes into metaframe and `shift₀` expression, and metaframe should handle effect of the `shift`. Also this metaframe decomposes into two metaframes separated by `reset₀` which is has same label.

```
data Metaframe (Δ : TContext) (Γ : Context) (T : Type) (Eff : Effects)
  : Type → Effects → TContext → ℕ → Set where
  mfempty : Metaframe Δ Γ T Eff T Eff Δ zero
  mfreset : ∀ {Δ' A B Eff' n l'}
    → (l : Label) → Δ ⊢ ttv l' ꞉e → Δ ; Γ ⊢ label l ꞉ (L ttv l' at B / Eff) / nil
    → (e : RExpr) → (Δ ; Γ , A ⊢ e ꞉ B / Eff)
    → Metaframe Δ Γ T (ttv l' :: Eff) A Eff' Δ' n
    ---------------------------------
    → Metaframe Δ Γ T Eff B Eff' Δ' n
  mframe : ∀ {A Eff' Δ' n B Eff'' Δ'' m}
    → Frame Δ     Γ A Eff B Eff' Δ' n
    → Metaframe Δ' Γ T Eff' A Eff'' Δ'' m
    ------------------------------------------
    → Metaframe Δ Γ T Eff B Eff'' Δ'' (n + m)
```

Metaframes, same as frames, can be lifted into arbitrary contexts, plugged and composed. They can also be composed with simple frames.

```
↑m : forall { Δ A B Eff Eff' Δ' n Γ' Γ}
  → Metaframe Δ Γ A Eff B Eff' Δ' n
  → Metaframe Δ (Γ' ⧺ Γ) A Eff B Eff' Δ' n
mplug : ∀ {Γ Δ Δ' T Eff A n E}
  → Metaframe Δ Γ T Eff A E Δ' n
  → (e : RExpr) → Δ' ; Γ ⊢ e ꞉ T / E
  → Σ[ res ∈ RExpr ] (Δ ; Γ ⊢ res ꞉ A / Eff)
_∘m_ : ∀ {Γ Δ Δ' Δ'' Eff Eff' Eff'' A B C n m}
  → Metaframe Δ Γ B Eff A Eff' Δ' n
  → Metaframe Δ' Γ C Eff' B Eff'' Δ'' m
  → Metaframe Δ Γ C Eff A Eff'' Δ'' (n + m)
_f∘m_ : ∀ {Γ Δ Δ' Δ'' Eff Eff' Eff'' A B C n m}
  → Frame Δ Γ B Eff A Eff' Δ' n
  → Metaframe Δ' Γ C Eff' B Eff'' Δ'' m
  → Metaframe Δ Γ C Eff A Eff'' Δ'' (n + m)
```

## 4.2 Reduction

Since labels need to be allocated, reduction relation is defined in terms of expression and state. State itself is just next label to be allocated. As frames are intrinsically typed, we need to provide judgment representing well-typedness of expressions.

```
infix 2 _↦_
State = ℕ
data _↦_ : RExpr × State → RExpr × State → Set where

  ↦new : ∀ {e s Δ E T A1 E1}
    → {te : `e (Data.Maybe.just s) Δ ;(∅ , L ttv zero at A1 / E1) ⊢ e ꞉ E / T}
    → new e ,′ s  ↦ new' s ( RExprSubstTyped._[_] e (label s) {te = te} {te1 = ⊢label Z} .proj₁ ) ,′ suc s

  β-lam-app : ∀ {e V s Δ Γ A B E}
    → {te : Δ ; (Γ , A) ⊢ e ꞉ B / E}
    → {tv : Δ ; Γ ⊢ V ꞉ A / E}
    → Value (lam e)
    → (v : Value V)
    → app (lam e) V ,′ s ↦ (proj₁ (RExprSubstTyped._[_] e V {te = te} {te1 = (gvalue v tv)})) ,′ s

  β-tlam-tapp : ∀ {k e T s}
      → Value (tlam k e)
      → tapp (tlam k e) T ,′ s ↦ e RExprSubst.e[t T ] ,′ s

  reset₀-vl : ∀ {V e' en s Δ Γ A B E }
      → ( v : Value V)
    → {tv : Δ ; Γ ⊢ V ꞉ A / E}
    → {ten : Δ ; (Γ , A) ⊢ en ꞉ B / E}
      → reset₀ V en e' ,′ s ↦ (RExprSubstTyped._[_] en V {te = ten} {te1 = (gvalue v tv)} .proj₁) ,′ s

  reset₀-k : ∀ {es en e' e s n Δ Δ' A T Eff Eff' B A' E' ls lr}
      → { f : Metaframe Δ ∅ T Eff A Eff' Δ' n }
      → ∀ {cont-type}
      → Value (e')
      --shift
```

```
    → {ts : Δ' ; ∅ ⊢ (shift₀ e' es) ∶ T / Eff'}
    → {tes : Δ' ; (∅ , T - Eff' > B) ⊢ es ∶ B / Eff' }
    → {tls : Δ' ; ∅ ⊢ e' ∶ (L ttv ls at B / Eff') / nil }
    → {tlvs : Δ' ⊢ ttv lr ∶e }
    --reset
    → {te : Δ ; ∅ ⊢ e ∶ A' / (ttv lr :: E') }
    → {ten : Δ ; ∅ , A' ⊢ en ∶ T / Eff }
    → {tlr : Δ ; ∅ ⊢ e' ∶ (L ttv lr at T / Eff) / nil }
    → {tlvr : Δ ⊢ ttv lr ∶e }
    → (proj₁ (mplug f (shift₀ e' es) ts)) ≡ e
    → reset₀ e en e' ,′ s
      ↦ RExprSubstTyped._[_] es
        (lam (reset₀ (proj₁ (mplug (↑m {Γ' = ∅ , T} f) (var 0) (⊢var Z))) en e'))
        {te = tes} {te1 = gvalue {E = Eff} vlam cont-type}
        .proj₁ ,′ s
```

Since simple reduction above is defined directly on redexes, we introduce -⟶ that represents reduction within metaframe. As we are only considering whole typed expressions, in place of Γ we use empty context.

```
infix 2 _-→_
data _-→_ : RExpr × State → RExpr × State → Set where
  -→frame : ∀ {e1 e1' e2 e2' s s' n Δ Δ' A T Eff Eff' t1 t2} → (f : Metaframe Δ ∅ T Eff A Eff' Δ' n)
    → e1' ,′ s ↦ e2' ,′ s'
    → Data.Product.proj₁ (mplug f e1' t1) ≡ e1
    → Data.Product.proj₁ (mplug f e2' t2) ≡ e2
    → (e1 ,′ s) -→ (e2 ,′ s')
```

## 4.3 Progress

```
data Decompose : State → (e : RExpr) → Set where
  de-simpl-redex : ∀ {e e2 s s' n Δ Δ' A T Eff Eff'}
    → (f : Metaframe Δ ∅ T Eff A Eff' Δ' n)
    → (e ,′ s) -→ (e2 ,′ s')
    → (t : Δ ; ∅ ⊢ e ∶ A / Eff)
    → Decompose s e
  de-shift : ∀ {s Δ Δ' T Eff A n Eff' es es' e l t}
    → (f : Metaframe Δ ∅ T Eff A Eff' Δ' n)
    → shift₀ (label l) es' ≡ es
    → Data.Product.proj₁ (mplug f es t) ≡ e
    → (t : Δ ; ∅ ⊢ e ∶ A / Eff)
    → (ts : Δ ; ∅ ⊢ es ∶ T / Eff')
    → Decompose s e
  de-val : ∀ {Δ Eff A s e} → { t : Δ ; ∅ ⊢ e ∶ A / Eff}
    → Value e
    → Decompose s e
data Progress : State → RExpr → Set where
  done : ∀ {e s} → Value e → Progress s e
  step : ∀ {e1 s1 e2 s2}
    → ( e1 ,′ s1 ) -→ ( e2 ,′ s2)
    → Progress s1 e1
```

Proof of progress would have a type of `progress : ∀ {A Δ Effs} → (s : State) → (e : RExpr) → (t : Δ ; ∅ ⊢ e ∶ A / Effs) → Progress s e`. In such proof We would use auxiliary struct `Decompose` which builder would walk down well typed expression recursively until it has reached either value, simple reduction (app, tapp, new), or shift, and return it with surrounding metaframe.

In case of shift, such metaframe by construction should have effect handler that has same effect as shift. So we can construct `rest₀-k` and surrounding metaframe. Other cases would either be immediate value, or simple reduction in context.

### 4.3.1 Preservation

Current definition of reduction relation would yield proof of preservation immediately if progress was given since, frames and expressions are well typed, and reduction relation requires proofs to plug into metaframes or subsitution.

# Chapter 5

# Machine

Here we define type erased expressions. And translation from previous `RExpr` type. Type abstraction is translated to normal abstraction, and type application is translated to application of identity function (it won't be used regardless of value).

```
data Erased : Set where
  var : ℕ → Erased
  lam : Erased → Erased
  app : Erased → Erased → Erased
  new : Erased → Erased
  shift₀ : Erased → Erased → Erased
  reset₀ : Erased → Erased → Erased → Erased
  label : ℕ → Erased

erased : Runtime.RuntimeExpr.RExpr → Erased
erased (RuntimeExpr.var x) = var x
erased (RuntimeExpr.lam x) = lam (erased x)
erased (RuntimeExpr.app x x₁) = app (erased x) (erased x₁)
erased (RuntimeExpr.tlam x x₁) = lam (erased x₁)
erased (RuntimeExpr.tapp x x₁) = app (erased x) (lam (var 0))
erased (RuntimeExpr.new x) = new (erased x)
erased (RuntimeExpr.new' x x₁) = erased x₁
erased (RuntimeExpr.shift₀ x x₁) = shift₀ (erased x) (erased x₁)
erased (RuntimeExpr.reset₀ x x₁ x₂) = reset₀ (erased x) (erased x₁) (erased x₂)
erased (RuntimeExpr.label x) = label x
```

Here we present draft of machine, together with draft of evaluation function.

```
Env : Set
data Val : Set
data Context : Set
MetaContext : Set

Counter = ℕ --label allocator
Env = List Val
data Val where
  thunk : Erased → Env → Val
  kont : Context → Val
  label : ℕ → Val
data Context where
  end : Context
  app₁ : Erased → Env → Context → Context
  app₂ : Val → Context → Context
  reset₀-label : Erased → Erased → Context → Context
  shift₀-label : Erased → Context → Context
MetaContext = List (Context × Maybe ℕ)

data State : Set where
  eval : Erased → Env → MetaContext → Counter → State
  cont : Context → MetaContext → Val → Counter → State
  val : Val → State
  err : State
```

```
init : Erased → State
init e = eval e [] ( (end ,′ Maybe.nothing) :: []) zero
{-
lookup : ∀ {A} ℕ → List A → Maybe A
lookup n xs = head(drop n xs)
move : State → State
move (eval (var x) ρ [] cnt) = err
move (eval (var x) ρ (x₁ :: mc) cnt) with lookup x ρ
... | Maybe.just v = cont (proj₁ x₁) mc (v) cnt
... | Maybe.nothing = err
move (eval (lam x) ρ [] cnt) = err
move (eval (lam x) ρ (x₁ :: mc) cnt) = cont (proj₁ x₁) mc (thunk x ρ) cnt
move (eval (app x x₁) ρ (c :: mc) cnt) = eval x ρ ( (app₁ x₁ ρ  (proj₁ c) ,′ Maybe.nothing) :: mc) cnt
move (eval (app x x₁) ρ [] cnt) = err
move (eval (new x) ρ mc cnt) = {!!}
move (eval (shift₀ x x₁) ρ mc cnt) = {!!}
move (eval (reset₀ x x₁ x₂) ρ mc cnt) = {!!}
move (eval (label x) ρ mc cnt) = {!!}
move (cont end mc v cnt) = cont end mc v cnt
move (cont (app₁ x x₁ c) mc v cnt) = eval x x₁ ((app₂ v c ,′ Maybe.nothing):: mc) cnt
move (cont (app₂ x c) mc v cnt) = {!!}
move (cont (reset₀-label x x₁ c) mc v cnt) = {!!}
move (cont (shift₀-label x c) mc v cnt) = {!!}
move err = err
move (val v) = (val v)

-}
```

# Chapter 6

# Discussion/Closing thoughts

In this paper we presented language with labeled delimited effect handlers together with partial formalisation of such language. We defined 3 term languages, typing judgements for 2 of them, translations between those languages and that typing is preserved. We defined intrinsically typed frames and metaframes that allow us to define reduction relation, how it preserves types. Since they are typed we statically know what effects are handled by frame. We also defined draft of abstract machine and how it could evaluate this language. This formalisation could be expanded and finished to fully prove correctness of the language and translations.

# Bibliography

[1] Guy Lewis Steele, Gerald Jay Sussman. The Revised Report on SCHEME: A Dialect of LISP. In MIT Artificial Intelligence Memo 452, 1978. URL: https://dspace.mit.edu/handle/1721.1/6283.

[2] Olivier Danvy and Andrzej Filinski. Abstracting control. In LISP and Functional Programming, pages 151-160, 1990. URL: https://dl.acm.org/doi/10.1145/91556.91622

[3] Matthias Felleisen. The theory and practice of first-class prompts. In POPL '88: Proceedings of the 15th ACM SIGPLAN-SIGACT symposium on Principles of programming languages, pages 180-190, 1988. URL: https://dl.acm.org/doi/10.1145/73560.73576

[4] Maciej Piróg, Piotr Polesiuk, Filip Sieczkowski. Typed Equivalence of Effect Handlers and Delimited Control. In 4th International Conference on Formal Structures for Computation and Deduction (FSCD 2019), pages 30:1-30:16, 2019. URL: https://doi.org/10.4230/LIPIcs.FSCD.2019.30

[5] Marek Materzok, Dariusz Biernacki.Subtyping Delimited Continuations. In ACM SIGPLAN Notices, Volume 46, Issue 9, pages 81-93, 2011. URL: https://dl.acm.org/doi/abs/10.1145/2034574.2034786